

Math 151A Homework 3

Tiya Chokhani
UCLA, Fall 2025

1.

Steffensen's method uses

$$p_0^{(0)}, \quad p_1^{(0)} = g(p_0^{(0)}), \quad p_2^{(0)} = g(p_1^{(0)}),$$

and then the accelerated value

$$p_0^{(1)} = p_0^{(0)} - \frac{(p_1^{(0)} - p_0^{(0)})^2}{p_2^{(0)} - 2p_1^{(0)} + p_0^{(0)}}.$$

We are given

$$p_0^{(0)} = 1, \quad p_2^{(0)} = 3, \quad p_0^{(1)} = 0.75,$$

and we want $p_1^{(0)}$.

Let $a = p_0^{(0)} = 1$, $b = p_1^{(0)}$, $c = p_2^{(0)} = 3$. Then

$$0.75 = 1 - \frac{(b-1)^2}{3-2b+1} = 1 - \frac{(b-1)^2}{4-2b}.$$

So

$$0.25 = \frac{(b-1)^2}{4-2b}.$$

Multiply:

$$0.25(4-2b) = (b-1)^2.$$

That is

$$\frac{4-2b}{4} = (b-1)^2 \implies 4-2b = 4(b-1)^2.$$

Expand:

$$4-2b = 4b^2 - 8b + 4 \implies 0 = 4b^2 - 6b = 2b(2b-3).$$

So $b = 0$ or $b = \frac{3}{2}$.

We take

$$p_1^{(0)} = \boxed{\frac{3}{2}}.$$

2.

Consider

$$f(x) = x^5 - 300x^2 - 126x + 5005.$$

We view the leading coefficient as a parameter a_5 and write

$$F(x, a_5) = a_5x^5 - 300x^2 - 126x + 5005.$$

Suppose $x = \alpha(a_5)$ is a root of $F(x, a_5) = 0$. Differentiate

$$F(\alpha(a_5), a_5) = 0$$

with respect to a_5 :

$$\frac{\partial F}{\partial a_5} + \frac{\partial F}{\partial x} \frac{d\alpha}{da_5} = 0.$$

We have

$$\frac{\partial F}{\partial a_5} = x^5, \quad \frac{\partial F}{\partial x} = 5a_5x^4 - 600x - 126.$$

At the unperturbed polynomial, $a_5 = 1$. We are told that $\alpha = 5$ is a root. Evaluate:

$$\left. \frac{\partial F}{\partial a_5} \right|_{x=5, a_5=1} = 5^5 = 3125,$$

$$\left. \frac{\partial F}{\partial x} \right|_{x=5, a_5=1} = 5(1) \cdot 5^4 - 600 \cdot 5 - 126 = 3125 - 3000 - 126 = -1.$$

Then

$$3125 + (-1) \frac{d\alpha}{da_5} = 0 \implies \frac{d\alpha}{da_5} = 3125.$$

For a small change $\Delta a_5 = \varepsilon$, the change in the root is approximately

$$\Delta \alpha \approx 3125 \varepsilon.$$

So the perturbed root is

$$\boxed{\alpha(\varepsilon) \approx 5 + 3125 \varepsilon.}$$

3.

We are given

$$f(x) = 676039x^{12} - 1939938x^{10} + 2078505x^8 - 1021020x^6 + 225225x^4 - 18018x^2 + 231.$$

All powers of x are even. Let $y = x^2$. Then $f(x) = 0$ is equivalent to $g(y) = 0$, where

$$g(y) = 676039y^6 - 1939938y^5 + 2078505y^4 - 1021020y^3 + 225225y^2 - 18018y + 231.$$

So instead of finding the 12 roots of $f(x)$ directly, we can find the 6 roots of $g(y)$, and then take $x = \pm\sqrt{y}$.

We can find the roots of $g(y)$ numerically using Newton's method and deflation. Below is a Python script that:

- runs Newton's method to approximate a root,
- deflates the polynomial to reduce its degree,
- repeats until all roots are found,
- then reconstructs the x -roots from the y -roots.

```
import numpy as np

def newton(poly, dpoly, x0, tol=1e-15, max_iter=100):
    x = x0
    for _ in range(max_iter):
        f = poly(x)
        df = dpoly(x)
        if df == 0:
            x = x + tol
            continue
        step = f/df
        x = x - step
        if abs(step) < tol:
            return x
    raise RuntimeError("no convergence")

def deflated_roots_via_recip(coeffs):
    g_coefs = np.array(coeffs, dtype=float)
    g_poly = np.poly1d(g_coefs)
    g_dpoly = np.polyder(g_poly)

    # reciprocal polynomial h(z) with reversed coeffs
    h_coefs = np.array(coeffs[::-1], dtype=float)

    roots_y_initial = []
    h_current = np.poly1d(h_coefs)

    guess = 10.0
```

```

for k in range(len(coeffs)-1):
    dh_current = np.polyder(h_current)
    try:
        z_root = newton(h_current, dh_current, guess)
    except RuntimeError:
        z_root = newton(h_current, dh_current, 1.0)

    y_root_est = 1.0 / z_root
    roots_y_initial.append(y_root_est)

    q, r = np.polydiv(h_current, np.poly1d([1.0, -z_root]))
    h_current = q
    guess = z_root

roots_y_polished = []
for y_guess in roots_y_initial:
    y_refined = newton(g_poly, g_dpoly, y_guess)
    roots_y_polished.append(y_refined)

return roots_y_polished

g_coeffs = [
    676039.0,
    -1939938.0,
    2078505.0,
    -1021020.0,
    225225.0,
    -18018.0,
    231.0
]

ys = deflated_roots_via_recip(g_coeffs)

x_roots = []
for val in ys:
    if val > 0:
        r = np.sqrt(val)
        x_roots.append(+r)
        x_roots.append(-r)

x_roots.sort()
print(x_roots)

```

The values that come out for y are

$$y = 1, \frac{1}{4}, \frac{1}{9}, \frac{1}{16}, \frac{1}{25}, \frac{1}{36}.$$

Therefore the 12 roots of $f(x)$ are

$$x = \pm 1, \pm \frac{1}{2}, \pm \frac{1}{3}, \pm \frac{1}{4}, \pm \frac{1}{5}, \pm \frac{1}{6}.$$

4.

Steffensen's method (Section 2.5) is used to accelerate a fixed point iteration $x_{n+1} = g(x_n)$ without needing to evaluate a derivative. We start with three successive evaluations

$$p_0^{(0)}, \quad p_1^{(0)} = g(p_0^{(0)}), \quad p_2^{(0)} = g(p_1^{(0)}),$$

and then define

$$p_0^{(1)} = p_0^{(0)} - \frac{(p_1^{(0)} - p_0^{(0)})^2}{p_2^{(0)} - 2p_1^{(0)} + p_0^{(0)}}.$$

A standard fixed point iteration is typically only linearly convergent. The Steffensen update above produces a new approximation $p_0^{(1)}$ that behaves like a Newton step and often shows near quadratic convergence in practice.

Although we talked about convergence acceleration in class, I wanted to look more closely at how Steffensen's update mimics a Newton step without ever computing a derivative, and why that can give near quadratic convergence in practice.

I chose this topic because it is a direct way to get Newton-like speed (fast convergence to the root) while only calling the original iteration function $g(x)$, with no need to compute or approximate $f'(x)$. This connects to the type of acceleration we used in Problem 1.